Mul 算子测试设计说明

1. 测试目标

本测试面向 math/mul 的端到端 example，用于提升 Mul 算子的：

- 功能正确性验证能力
- op_api / op_host 层覆盖率
- 结果校验有效性
- 异常输入分支覆盖

重点覆盖 4 个对外 API：

- aclnnMul
- aclnnMuls
- aclnnInplaceMul
- aclnnInplaceMuls

其中：

- aclnnMul / aclnnInplaceMul 主要覆盖 tensor-tensor 分支
- aclnnMuls / aclnnInplaceMuls 主要覆盖 tensor-scalar 分支
- 不同 dtype / shape / 边界值用于触发不同的类型提升、广播和 tiling 路径

---

2. 测试策略

2.1 选择端到端测试的原因

官方 example 仅演示调用流程，不能验证结果正确性。 因此本测试在 example 基础上补充：

1. 构造输入 tensor / scalar
2. 调用 GetWorkspaceSize + Execute
3. 回拷输出
4. 在 CPU 端独立计算期望值
5. 逐元素比较结果

这样既能进入真实执行链路，也能保证测试不是“只打印不校验”。

2.2 用例组织策略

测试分为两类：

（1）正向执行用例

覆盖：

- 正常同 shape
- 广播
- 空 tensor
- 大 tensor
- 非连续 tensor
- 特殊数值
- 多 dtype
- 四类 API 变体

（2）异常输入用例

覆盖：

- nullptr
- shape 不可广播

- 输出 shape 错误
- 输出 dtype 错误
- 不支持 dtype
- 不支持混合 dtype
- scalar API 的错误参数

---

3. 用例设计覆盖面

3.1 API 覆盖

本测试覆盖了全部 4 个 API：

API	覆盖内容
aclnnMul	同 shape、广播、空 tensor、大 tensor、特殊 dtype、特殊值
aclnnMuls	float / int / fp16 / bf16 标量乘
aclnnInplaceMul	广播与同 shape 原地乘
aclnnInplaceMuls	float / int 原地标量乘

3.2 dtype 覆盖

已覆盖的 dtype 包括：

- ACL_FLOAT
- ACL_FLOAT16
- ACL_BF16
- ACL_DOUBLE
- ACL_INT8
- ACL_UINT8
- ACL_INT16
- ACL_INT32
- ACL_INT64
- ACL_BOOL

覆盖目的：

- FLOAT / FLOAT16 / BF16 / DOUBLE：覆盖浮点与混合精度路径
- INT8 / UINT8：覆盖饱和截断路径
- INT16 / INT32 / INT64：覆盖整数计算路径
- BOOL：覆盖布尔类型特殊处理路径

同时设计了非法 dtype：

- ACL_UINT32

用于覆盖不支持类型的参数校验分支。

3.3 shape 覆盖

覆盖了以下 shape 组合：

- 同 shape：如 {4,2} * {4,2}
- 广播：如 {2,3} * {3}
- 高维广播：如 {2,1,3} * {1,4,1}

- 空 tensor：如 {0,3}
- 较大 tensor：如 {32,64}
- 非连续 tensor：通过自定义 stride 构造 view tensor

设计目的：

- 覆盖 op_host 中的 shape 推断逻辑
- 覆盖广播路径
- 覆盖 tiling 对大 tensor / 非连续布局的处理

3.4 数值边界覆盖

覆盖了以下数值边界：

- 零值
- 负数
- 大小混合正负值
- NaN
- Inf
- -Inf
- double 极值 / denorm
- int8 / uint8 饱和截断场景

设计目的：

- 验证结果正确性
- 覆盖 kernel / 调度中的特殊数值路径
- 提高精度问题发现能力

---

4. 结果验证方案

4.1 CPU 参考计算

所有正向用例均在 CPU 侧独立计算期望值，公式为：

- tensor-tensor：expect[i] = self[i] * other[i]
- tensor-scalar：expect[i] = self[i] * scalar
- inplace：以输入 self 的 shape 为输出 shape 进行相同计算

广播场景在 CPU 侧按广播规则手工索引计算。

4.2 精度比较策略

对不同 dtype 使用不同比较方式：

- FLOAT：atol=rtol=1e-5
- FLOAT16：atol=rtol=1e-3
- BF16：atol=rtol=1e-2
- DOUBLE：atol=rtol=1e-12
- 整数与 BOOL：精确匹配

同时对：

- NaN
- Inf
- -Inf

做专门判断，避免普通误差比较失效。

---

5. 覆盖率设计思路分析

5.1 对 op_api/aclnn_mul.cpp 的覆盖思路

该文件负责：

- 参数校验
- 类型提升
- API 分流
- 连续 / 非连续路径
- 部分平台 / dtype 路由

因此测试从以下维度设计：

- 四类 API 都调用
- 覆盖多种 dtype 组合
- 覆盖广播与非广播
- 覆盖非连续 tensor
- 覆盖 nullptr / 不支持 dtype / 输出 shape/dtype 错误

5.2 对 op_host/mul_tiling_arch35.cpp 的覆盖思路

该文件主要依赖：

- dtype 组合
- shape 大小
- tensor 连续性
- 是否需要广播

因此设计了：

- large tensor
- broadcast
- 空 tensor
- 非连续 tensor
- 多 dtype tensor-tensor / tensor-scalar 用例

---

6. 当前覆盖情况分析

6.1 已覆盖较好的代码

通常已较好覆盖：

- aclnnMul 主路径
- aclnnMuls 主路径
- aclnnInplaceMul / aclnnInplaceMuls 基础路径
- 广播 shape 推断
- 多种基础 dtype dispatch
- 空指针和明显非法输入分支
- float / fp16 / bf16 / int / bool / double 正常结果验证

6.2 可能未完全覆盖的代码

仍可能存在未完全覆盖的部分包括：

- 平台条件相关的特殊分支
- 某些仅在特定 SoC / 特定寄存器模式下可进入的路径
- 部分极端 dtype 组合对应的低频逻辑

- kernel 内部具体计算路径（CPU 模拟器下通常无法通过 gcov 完整统计）

6.3 未覆盖原因说明

未覆盖部分主要原因有三类：

1. **运行环境限制**
    - 某些路径依赖特定 SoC 或硬件能力
    - CPU 模拟器下无法自由切换平台条件
2. **example 测试职责边界**
    - 本测试是端到端 example，不直接 stub host/api 内部对象
    - 无法像 UT 那样强制制造某些内部状态
3. **kernel 层不可观测**
    - kernel 在模拟器内部执行，gcov 对其不可见
    - 因此测试重点放在 op_api + op_host

---

7. 质量分析

本测试相比官方示例的改进点：

- 增加了完整的结果校验，而非仅打印输出
- 覆盖了 4 个 API，而非仅 1 条主路径
- 覆盖了多种 dtype、广播、空 tensor、大 tensor、非连续 tensor
- 增加了大量异常输入分支
- 输出统一使用 [PASS] / [FAIL] 和汇总结果，便于评测与定位

整体上，该测试兼顾了：

- 编译运行稳定性
- 功能正确性
- 覆盖率提升
- 精度验证有效性

适合作为 Mul 算子的高质量端到端测试提交。